

Development Environment Setup

1. GITHUB ACCOUNT CREATION

Let's start with GitHub.

Prerequisites

- A valid email address.
- A modern web browser (Google Chrome or Microsoft Edge recommended).
- A stable Internet connection.

Step 1 – Open GitHub

Open your browser and navigate to:

<https://github.com>

Do **not** sign in yet if you don't already have an account.

Step 2 – Create Account

Click:

Sign up

You should be prompted for:

- Email address
- Password
- Username

Step 3 – Username

Choose a professional username.

Examples:

csivanandareddy
sivanandareddy
csreddy
sivananda-reddy
firstname-lastname

Avoid:

coolboy123
dragon999
killer007
tinymking

GitHub is part of your professional engineering portfolio.

Step 4 – Email Verification

GitHub will send a verification email.

Verify the email before continuing.

Step 5 – First Login

After verification,

Log in.

Participants should now see:

- Their profile icon
- Dashboard
- Repositories tab

Step 6 – Verification

Verify:

✓ Logged in successfully

✓ Email verified

✓ Username visible

✓ Dashboard accessible

Step 7 – Create First Repository

Create a repository.

Repository Name: **TinyML_Workshop**

Other settings:

Public

✓ Add README

Do not add:

- .gitignore
- License

Click

Create Repository

Step 8 – Final Verification

Participants should now have:

TinyML_Workshop

README.md

visible in GitHub.

Common Issues

- Username already exists.
- Verification email delayed.
- Spam folder.
- Weak password.
- Corporate firewall blocks GitHub.

Troubleshooting

- Try another username.

- Check Spam/Junk.
- Use a personal email if the institutional email blocks verification.
- Use Chrome or Edge if browser compatibility issues occur.

Checkpoint

Before proceeding, confirm that you:

- Can sign in to GitHub.
- Can access the **TinyML_Workshop** repository.
- Has successfully created the repository with a README.md file.

2. STMICROELECTRONICS ACCOUNT

Prerequisites

Before beginning, ensure that participants have:

- A valid email address.
- A modern web browser (Google Chrome or Microsoft Edge recommended).
- A stable Internet connection.

Step 1 – Open the ST Website

Navigate to the official STMicroelectronics website:

<https://www.st.com>

Step 2 – Create an Account

Click **Sign In** (top-right corner), then select **Create an Account**.

You will be prompted to enter:

- First Name
- Last Name
- Email Address
- Password
- Country/Region
- Company/Organization (if applicable)

Step 3 – Accept Terms

Read and accept the Terms and Conditions and Privacy Policy.

Complete any CAPTCHA verification if prompted.

Step 4 – Verify Email

An email verification link will be sent.

- Open your email inbox.
- Click the verification link.
- Return to the ST website.

If the email is not received within a few minutes, check the Spam/Junk folder.

Step 5 – Sign In

After verification, sign in using the newly created credentials.

Step 6 – Verify Access

Confirm that you can:

- Successfully sign in.
- Access the **MyST** dashboard.
- Open the STM32 software download pages.

- Search for **STM32CubeIDE**.

Checkpoint

verify that you:

- ✓ Can sign in successfully.
- ✓ Can access the STM32CubeIDE download page.
- ✓ Can browse the STM32Cube software catalog.

Do **not** proceed until these checkpoints are complete.

Common Issues

Issue	Possible Cause	Resolution
Verification email not received	Email delay or spam filtering	Check Spam/Junk folder; wait a few minutes and resend if needed.
Unable to sign in after verification	Incorrect password or email	Reset the password or verify the email address entered.
Download page inaccessible	Not signed in	Ensure the participant is logged into their ST account.
Corporate or institutional network restrictions	Firewall or proxy	Use a different network or consult the network administrator.

3. INSTALLING VISUAL STUDIO CODE

Step 1 – Open the Official Website

Navigate to:

<https://code.visualstudio.com>

Step 2 – Download

Select the installer appropriate for your operating system.

For this workshop:

Windows 64-bit (User Installer) is recommended.

Step 3 – Install

Run the installer.

During installation, **enable the following options**:

- ✓ Add "Open with Code" to the Windows Explorer context menu.
- ✓ Add "Open with Code" to the directory context menu.
- ✓ Register Code as an editor for supported file types.
- ✓ Add VS Code to the system PATH (recommended).

Accept the remaining default settings and complete the installation.

Step 4 – Launch VS Code

Open Visual Studio Code.

Verify that:

- The application starts without errors.
- The Welcome screen is displayed.
- The Activity Bar, Explorer, and integrated Terminal are available.

Step 5 – Create the Workshop Folder

Create the following directory structure:

```
TinyML_Workshop/  
|  
├─ Python/  
├─ STM32/  
├─ Datasets/  
├─ Models/  
├─ ONNX/  
├─ Documentation/  
└─ Utilities/
```

Open the **TinyML_Workshop** folder in VS Code using **File → Open Folder**.

Step 6 - Configure the Default Integrated Terminal

1. Open Visual Studio Code.
2. Press **Ctrl + Shift + P**.

3. Type:
Terminal: Select Default Profile
4. Select
Command Prompt
5. Open a new terminal using
Terminal → New Terminal
Expected result:
D:\TinyML_Workshop>
(or the your's own project directory)
6. Verification
Execute
`echo TinyML Workshop Ready!`
Expected output
TinyML Workshop Ready!

Checkpoint

Before continuing, verify that you:

- ✓ Have installed VS Code successfully.
- ✓ Can open the workshop folder.
- ✓ Can access the integrated terminal.
- ✓ Can create and save a file.

Common Issues

Issue	Possible Cause	Resolution
VS Code does not launch	Incomplete installation	Reinstall using the official installer.
"Open with Code" missing	Context menu options not selected	Re-run the installer and enable the Explorer integration options.
Terminal does not open	Terminal profile misconfiguration	Restart VS Code and verify the default terminal profile.
Folder not opening	Insufficient permissions	Choose a folder inside the user's Documents directory or another writable location.
PowerShell terminal opens but keyboard input is not accepted	PowerShell profile or Windows configuration issue	Change the default integrated terminal to Command Prompt
'python' is not recognized	Python PATH not configured	Reinstall Python and enable Add Python to PATH
'git' is not recognized	Git PATH missing	Reinstall Git and select Git from the command line

Issue	Possible Cause	Resolution
VS Code terminal opens in the wrong folder	Incorrect workspace	Open the TinyML_Workshop folder before creating a new terminal
ST account download unavailable	Not signed in	Log in to the STMicroelectronics account before downloading software

4. INSTALLING AND CONFIGURING GIT

Step 1

Open the Git download page.

<https://git-scm.com/downloads>

Download the latest **64-bit Git for Windows** installer.

Step 2

Run the installer.

Accept the default installation options except for the following.

Editor

Choose “Visual Studio Code”

PATH Environment

Choose “Git from the command line and also from 3rd-party software”

HTTPS Transport Backend

Leave the default option “Use the OpenSSL library”

Line Ending Conversion

Choose “Checkout Windows-style, commit Unix-style line endings”

Terminal Emulator

Choose “Use Windows' default console window”

This ensures Git works properly inside the VS Code Command Prompt terminal.

Default Branch Name

Choose “Override the default branch name” by **main**

This avoids later confusion because GitHub repositories use **main** by default.

Remaining Options

Leave everything else as the default.

Click Install

Step 3

After installation completes, Click Finish

Checkpoint

You should now have Git installed.

Step 4 - Verifying Git Installation

Open

Visual Studio Code.

Open

Terminal → New Terminal

Verify that the terminal opens as Command Prompt (cmd)

Run

```
git --version
```

Expected output

```
git version 2.xx.x.windows.x
```

If Git is not recognized,
Restart VS Code.

If the problem still exists,
Restart Windows.

Checkpoint

✓ Git version displayed.

Step 5 – Configuring Git

Git stores your name and email inside every commit.

Run

```
git config --global user.name "Your Name"
```

Example: `git config --global user.name "Sivananda Reddy"`

Run

```
git config --global user.email "your_email@gmail.com"
```

Example: `git config --global user.email "abc@gmail.com"`

Verify the configuration.

```
git config --global --list
```

Expected output

```
user.name=Sivananda Reddy
```

```
user.email=abc@gmail.com
```

These details become part of every Git commit and identify the author of the changes.

Checkpoint

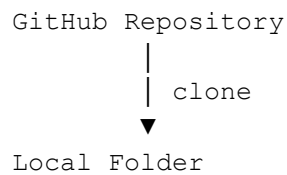
You should successfully configure:

- user.name
- user.email

Step 6 – Cloning the Workshop Repository

Dont confuse a **GitHub repository** with a **local folder**.

Remember:



Step 6.1

Choose a location for workshop files.

Example

D:\Workshop

Step 6.2

Open Terminal.

Navigate to that location.

```
D:
cd \
mkdir Workshop
cd Workshop
```

Step 6.3

Copy your repository URL.

Example

https://github.com/username/TinyML_Workshop.git

Step 6.4

Clone it.

```
git clone https://github.com/username/TinyML_Workshop.git
```

Expected output

Cloning into 'TinyML_Workshop'...

Step 6.5

Move inside the repository.

```
cd TinyML_Workshop
```

Step 6.6

Open it.

```
code .
```

Checkpoint

Everyone should now have the repository open in VS Code.

Step 7 – Creating the Workshop Folder Structure

Create the following folders.

```
TinyML_Workshop/
  Datasets
  Documentation
  Models
  ONNX
```

```
Python
STM32
Utilities
```

Git cannot store empty folders.

Therefore,

create a **.gitkeep** inside every folder.

Verify Git status.

```
git status
```

Expected output

Untracked files

Checkpoint

Folders visible.

Step 8 – First Commit

Stage all files.

```
git add .
```

Verify

```
git status
```

Expected output

Changes to be committed

Commit

```
git commit -m "Create workshop directory structure"
```

Push

```
git push
```

Because the repository was cloned, Git already knows where to push.

Refresh GitHub.

The folders should now appear online.

Checkpoint

Every participant should have:

- ✓ Git installed
- ✓ Git configured
- ✓ Repository cloned
- ✓ Folder structure created
- ✓ README files added
- ✓ First commit completed
- ✓ GitHub updated

5 – INSTALLING PYTHON

Step 1 - Downloading Python

Visit the official Python website:

<https://www.python.org/downloads/>

Download the latest **Python 3.x (64-bit)** Windows installer.

Step 2 – Installing Python

Run the installer.

Important

Before clicking **Install Now**, enable:

☒ **Add Python to PATH**

This is the most important option in the installer.

Then click: **Install Now**

Wait for the installation to complete.

Finally, click: **Close**

Step 3 – Verifying Python Installation

Open the **Command Prompt terminal** inside VS Code.

Execute:

```
python --version
```

Expected output:

```
Python 3.x.x
```

Then execute:

```
python
```

Expected output:

```
>>>
```

This is the Python Interactive Interpreter (REPL).

Exit using:

```
exit()
```

or press:

```
Ctrl + Z
```

Enter

Step 4 – Creating the First Python Program

Inside the **Python** folder, create: `hello.py`

Add:

```
print("Hello TinyML!")
```

Save the file.

Execute:

```
cd Python
```

```
python hello.py
```

Expected output:

Hello TinyML!

Checkpoint

Verify that every participant can:

- Run `python --version`.
- Start the Python interpreter.
- Exit the interpreter.
- Execute `hello.py`.

Common Issues

Issue	Possible Cause	Resolution
'python' is not recognized	Python not added to PATH	Reinstall Python and enable Add Python to PATH .
Wrong Python version	Multiple Python installations	Select the correct interpreter in VS Code.
hello.py does not execute	Incorrect working directory	Navigate to the Python folder before running the script.
VS Code does not recognize Python	Python extension missing	Install the Microsoft Python extension (covered in the next section).

6. CONFIGURING VISUAL STUDIO CODE FOR PYTHON DEVELOPMENT

Step 1 – Installing the Python Extension

Open VS Code.

Step 2

Click

Extensions

or press

Ctrl + Shift + X

Step 3

Search

Python

Step 4

Install

Python Published by Microsoft

Verify that the publisher is **Microsoft**.

Step 5

Wait until installation finishes.

Verification

Observe

Installing...

becomes

Installed

Checkpoint

Everyone should have

 Microsoft Python Extension installed.

Step 6 – Installing Pylance

In the Extensions itself, search

Pylance

Install

Pylance

Publisher

Microsoft

Verification

Status

Installed

Checkpoint

Everyone has

✓ Python Extension

✓ Pylance

Step 7 – Selecting the Python Interpreter

This is one of the most important activities.

Step 7.1

Open `hello.py`

Step 7.2

Press

Ctrl + Shift + P

Step 7.3

Search

Python: Select Interpreter

Click it.

Step 7.4

VS Code displays available interpreters.

You may see something similar to

Python 3.14.6

Select

Python 3.14.6 (64-bit)

Verification

Bottom-right corner of VS Code should display

Python 3.14.6 64-bit

Step 8 – Verifying Syntax Highlighting

Open

`hello.py`

Observe

```
print("Hello TinyML!")
```

The word

```
print
```

should appear in a different colour.

Strings should appear in another colour.

Verification

Syntax colouring is active.

Step 10 – Verifying IntelliSense

Modify

```
pri
```

Pause.

Observe

```
print()
```

appears automatically.

Press Tab

or

Enter

VS Code completes

```
print()
```

Now type

```
import math
```

Go to next line.

Type

```
math.
```

Observe

A large list appears

```
sin  
cos  
sqrt  
log  
...
```

Verification

Autocomplete works.

Step 11 – Running Python from VS Code

Click

► Run Python File

Observe

Terminal automatically opens.

Output

```
Hello TinyML!
```

Now compare with

```
python hello.py
```

Explain

Both execute exactly the same Python interpreter.

The only difference is

The Python Extension automatically builds the command.

Verification

Both execution methods work.

Step 12 – First Debugging Session

This is extremely useful.

Modify code

```
name = "TinyML"
print(name)
print("Workshop")
```

Click Left side of line `print(name)`

A red dot appears.

Breakpoint created.

Click

Run → Start Debugging

or

Press

F5

Step 12.1

Click

Python Debugger (the first option)

Do not select:

- More Node.js options
- Install an extension for Python

You already have the required extension installed.

Step 12.2

After clicking **Python Debugger**, it may ask you to select a debug configuration.

Choose:

Python File

(or **Python Debugger: Python File**, depending on the version).

This tells VS Code:

"Run and debug the currently opened Python file."

Step 12.3

The debugger should now start.

Execution will stop at your breakpoint (the red dot on line 3).

You should notice:

- A yellow arrow pointing to the current line.
- The program paused before executing `print(name)`.
- A Debug toolbar appearing at the top.

Step 12.4

On the Debug toolbar, you'll see buttons like:

- ▶ Continue
- ⏮ Step Over
- ↓ Step Into
- ↑ Step Out
- ↺ Restart
- Stop

For this simple example:

1. Click **Step Over** (⏮).
2. Observe that `print(name)` executes.
3. Click **Step Over** again.
4. Observe that `print("Workshop")` executes.
5. Click **Continue** (▶) or **Stop** (■).

Final Verification

Every participant should verify

- ✓ Python Extension installed
- ✓ Pylance installed
- ✓ Interpreter selected
- ✓ Syntax highlighting working
- ✓ IntelliSense working
- ✓ Run button working
- ✓ Terminal execution working
- ✓ Debugger working

Common Issues

Problem	Solution
Python interpreter not listed	Restart VS Code
Run button missing	Install the Microsoft Python Extension
No IntelliSense	Install Pylance
Wrong interpreter selected	Use Python: Select Interpreter
python not recognized	Verify PATH and restart VS Code
Debugging fails	Ensure <code>hello.py</code> is saved before starting the debugger

Step 13 – Exploring the Python Environment

Run

```
python
```

Then:

```
import sys
```

```
print(sys.version)
print(sys.executable)
exit()
```

This shows:

- The exact Python version in use.
- The full path to the interpreter.
- That the interpreter selected in VS Code matches the one available in the terminal.

7 – CREATING THE PYTHON VIRTUAL ENVIRONMENT

Step 1 – Verify Current Python

Open the VS Code terminal.

Run:

```
python --version
```

Expected output:

```
Python 3.14.6
```

Next:

```
where python
```

This confirms the interpreter currently in use.

Step 2 – Create the Virtual Environment

Ensure the terminal is in the project root.

Example:

```
D:\Workshop\TinyML_Workshop
```

Create the environment:

```
python -m venv .venv
```

Wait a few seconds.

A new folder appears:

```
TinyML_Workshop
|
├── .venv
├── Python
├── STM32
├── Models
...

```

.venv contains a private Python installation for this project.

Step 3 – Activate the Virtual Environment

On Windows Command Prompt:

```
.venv\Scripts\activate
```

The terminal prompt should change from:

```
D:\Workshop\TinyML_Workshop>
```

to:

```
(.venv) D:\Workshop\TinyML_Workshop>
```

Every package installed from now on will go into this virtual environment.

Step 4 – Verify the Interpreter

Run:

```
where python
```

Now the first entry should point into the virtual environment, for example:

```
D:\Workshop\TinyML_Workshop\.venv\Scripts\python.exe
```

Then run:

```
python --version
```

It should still display the same Python version, but now it is executing from the virtual environment.

Step 5 – Configure VS Code

Open the Command Palette:

Ctrl + Shift + P

Run:

Python: Select Interpreter

VS Code should detect:

.venv (Python 3.14.6)

Select it.

Verify the status bar now shows the .venv interpreter.

Step 6 – Verify the Environment

Create a file:

```
Python\verify_environment.py
```

Add:

```
import sys
print("Python Version:", sys.version)
print("Executable:", sys.executable)
```

Run:

```
python Python\verify_environment.py
```

Expected output:

```
Python Version: 3.14.6
```

```
Executable: ...\\TinyML_Workshop\\.venv\\Scripts\\python.exe
```

This confirms that VS Code and the terminal are using the same interpreter.

Step 7 – Deactivate the Environment

Run:

```
deactivate
```

The prompt should return to:

```
D:\\Workshop\\TinyML_Workshop>
```

The virtual environment can be reactivated whenever we continue working on the project.

Checkpoint

Verify that you can:

- Create a virtual environment.

- Activate it.
- Observe the (.venv) prompt.
- Select the .venv interpreter in VS Code.
- Confirm the interpreter path using `sys.executable`.
- Deactivate the environment.

8 – INSTALLING THE SCIENTIFIC PYTHON ECOSYSTEM

Step 1 – Verify Virtual Environment

Before installing anything, ensure the virtual environment is active.

The terminal prompt should look similar to:

```
(.venv) D:\TinyML_Workshop>
```

If not, activate it:

```
.venv\Scripts\activate
```

Verify:

```
python --version
```

Expected:

```
Python 3.14.6
```

Step 2 – Upgrade pip

Run:

```
python -m pip install --upgrade pip
```

Verify:

```
pip --version
```

Expected:

```
pip xx.x.x
```

Step 3 – Install NumPy

Install:

```
pip install numpy
```

Verify:

```
python
import numpy as np
a = np.array([1,2,3,4])
print(a)
print(a*2)
exit()
```

Expected output:

```
[1 2 3 4]
```

```
[2 4 6 8]
```

Step 4 – Install Pandas

Install:

```
pip install pandas
```

Verification:

```
import pandas as pd
df = pd.DataFrame({
    "Sensor": ["Temp", "Humidity", "Pressure"],
    "Value": [30, 62, 1013]
})
```



```
print(df)
```

Expected:

	Sensor	Value
0	Temp	30
1	Humidity	62
2	Pressure	1013

Later, CSV datasets from sensors will be loaded using Pandas.

Step 5 – Install Matplotlib

Install:

```
pip install matplotlib
```

Create:

```
Python\plot_test.py
```

```
import matplotlib.pyplot as plt
x=[1,2,3,4,5]
y=[2,4,6,8,10]
plt.plot(x,y)
plt.title("TinyML Test Plot")
plt.show()
```

Run:

```
python Python\plot_test.py
```

Expected:

A graph window appears with a straight line.

Step 6 – Install SciPy

Install:

```
pip install scipy
```

Verification:

```
from scipy import stats
print(stats.norm.pdf(0))
```

Expected output:

Approximately: 0.39894228

Step 7 – Install Jupyter Notebook

Install:

```
pip install notebook
```

Verify:

```
jupyter notebook
```

Expected:

- Browser opens.
- Jupyter dashboard appears.

- Terminal displays a local URL.

Stop the server with:

Ctrl + C

Confirm with:

y

Step 8 – Verify Installed Packages

Run:

```
pip list
```

Participants should see packages such as:

```
numpy
pandas
matplotlib
scipy
notebook
pip
setuptools
```

Step 9 – Comprehensive Verification Script

Create:

```
Python\scientific_stack_test.py

import numpy
import pandas
import matplotlib
import scipy

print("NumPy Version      :", numpy.__version__)
print("Pandas Version      :", pandas.__version__)
print("Matplotlib Version:", matplotlib.__version__)
print("SciPy Version        :", scipy.__version__)

print("\nScientific Python Environment Ready!")
```

Run:

```
python Python\scientific_stack_test.py
```

Expected:

```
NumPy Version   :...
Pandas Version  :...
Matplotlib Version: ...
SciPy Version   :...
Scientific Python Environment Ready!
```

Final Verification Checklist

You should confirm:

- ☒ Virtual environment is active.
- ☒ pip upgraded successfully.
- ☒ NumPy installed and verified.
- ☒ Pandas installed and verified.
- ☒ Matplotlib installed and displayed a graph.
- ☒ SciPy installed and imported successfully.
- ☒ Jupyter Notebook launched.
- ☒ pip list displays all installed packages.
- ☒ Comprehensive verification script runs without errors.

9 – INSTALLING THE DEEP LEARNING FRAMEWORK (PYTORCH)

Step 1 – Verify Active Virtual Environment

Verify the virtual environment is active:

```
python --version
where python
```

Ensure the interpreter points to:

```
...\TinyML_Workshop\venv\Scripts\python.exe
```

Step 2 – Install PyTorch

Install

```
pip install torch torchvision torchaudio
```

Step 3 – Verify Installation

Run:

```
pip show torch
```

Expected

- Package name
- Version
- Installation location

Step 4 – Verify PyTorch Version

Create:

```
Python\verify_pytorch.py
```

```
import torch
print("PyTorch Version :", torch.__version__)
print("CUDA Available :", torch.cuda.is_available())
```

Run:

```
python Python\verify_pytorch.py
```

Expected:

```
PyTorch Version : 2.x.x
CUDA Available : False
```

Step 5 – First Tensor

Modify:

```
import torch
a = torch.tensor([1,2,3])
print(a)
```

Expected:

```
tensor([1, 2, 3])
```

Step 6 – First Neural Network

Modify:

```
import torch
import torch.nn as nn

model = nn.Sequential(
    nn.Linear(4, 8),
    nn.ReLU(),
    nn.Linear(8, 2)
)

print(model)
```

You will see the network architecture printed.

Step 7 – Dummy Inference

Modify:

```
import torch
import torch.nn as nn

model = nn.Sequential(
    nn.Linear(4, 8),
    nn.ReLU(),
    nn.Linear(8, 2)
)

x=torch.rand(1,4)

y=model(x)

print(y)
```

Expected:

A tensor of shape:
torch.Size([1,2])

Step 8 – Verify TorchVision

```
import torchvision
print(torchvision.__version__)
```

Explain:

TorchVision provides:

- Datasets
- Image transforms
- Pretrained vision models

Step 9 – Verify TorchAudio

```
import torchaudio
print(torchaudio.__version__)
```

Explain:

TorchAudio supports:

- Audio datasets
- Speech preprocessing
- Audio feature extraction

Step 10 – Comprehensive Verification

Create:

```
Python\deep_learning_stack_test.py

import torch
import torchvision
import torchaudio

print("Torch      :", torch.__version__)
print("TorchVision :", torchvision.__version__)
print("TorchAudio  :", torchaudio.__version__)

print("CUDA Available :", torch.cuda.is_available())

x=torch.rand(2,3)

print("\nTensor Test")

print(x)

print("\nDeep Learning Environment Ready!")
```

Run:

```
python Python\deep_learning_stack_test.py
```

Expected:

- Versions displayed.
- CUDA status.
- Random tensor.
- Final success message.

Final Verification Checklist

You should confirm:

-  PyTorch installed.
-  TorchVision installed.

-  TorchAudio installed.
-  Version verification successful.
-  Tensor operations executed.
-  Simple neural network created.
-  Forward inference completed.
-  Comprehensive verification script executed successfully.

10 – INSTALLING THE ONNX ECOSYSTEM

Step 1 – Verify Virtual Environment

First ensure the virtual environment is active.

```
python --version
```

```
where python
```

Verify that the interpreter is from:

```
...\TinyML_Workshop\.venv\
```

Step 2 – Install ONNX

Install:

```
pip install onnx
```

Verification

Run

```
pip show onnx
```

You should see:

- Name
- Version
- Installation path

Step 3 – Install ONNX Runtime

Install:

```
pip install onnxruntime
```

Verify:

```
pip show onnxruntime
```

Step 4 – Install ONNX Simplifier

Install:

```
pip install onnxsim
```

Verify:

```
pip show onnxsim
```

Step 5 – Install Netron

Netron is not a Python package. It is a standalone model visualization tool.

There are two options:

Option 1 (Recommended)

Visit:

<https://netron.app>

This browser-based viewer requires no installation.

Option 2

Install locally:

```
pip install netron
```

Run:

```
netron
```

The browser opens automatically.

Step 6 – Verify ONNX

Create:

```
Python\verify_onnx.py
```

```
import onnx
print("ONNX Version:", onnx.__version__)
```

Run:

```
python Python\verify_onnx.py
```

Expected:

ONNX Version: ...

Step 7 – Verify ONNX Runtime

Create:

```
Python\verify_onnxruntime.py
```

```
import onnxruntime as ort

print("ONNX Runtime Version:", ort.__version__)
```

Run:

```
python Python\verify_onnxruntime.py
```

Expected:

ONNX Runtime Version: ...

Step 8 – Export the First ONNX Model

Create:

```
Python\export_onnx.py
```

```
import torch
import torch.nn as nn
```

```

model = nn.Sequential(
    nn.Linear(4,8),
    nn.ReLU(),
    nn.Linear(8,2)
)

dummy_input = torch.randn(1,4)

torch.onnx.export(
    model,
    dummy_input,
    "simple_model.onnx",
    input_names=["input"],
    output_names=["output"]
)

print("ONNX model exported successfully.")

```

Run:

```
python Python\export_onnx.py
```

Expected:

ONNX model exported successfully.

Verify that:

```
simple_model.onnx
    appears in the project folder.
```

IT GIVES ERROR.

FIX IT:

Simply install the missing package inside the activated virtual environment:

```
pip install onnxscript
```

After it completes, verify:

```
pip show onnxscript
```

Then rerun

```
python Python\export_onnx.py
```

It should now successfully create:

```
simple_model.onnx
```

Step 9 – Open the Model in Netron

Open:

```
simple_model.onnx  
using Netron.
```

You should observe:

- Input node
- Linear layer
- ReLU
- Linear layer
- Output node

Step 10 – Load the ONNX Model

Create:

```
Python\load_onnx.py  
  
import onnx  
  
model = onnx.load("simple_model.onnx")  
  
onnx.checker.check_model(model)  
  
print("ONNX model is valid.")
```

Run:

```
python Python\load_onnx.py
```

Expected:

```
ONNX model is valid.
```

Step 11 – Run ONNX Runtime Inference

Create:

```
Python\onnx_inference.py  
  
import numpy as np  
import onnxruntime as ort  
  
session = ort.InferenceSession("simple_model.onnx")  
input_name = session.get_inputs()[0].name  
x = np.random.randn(1,4).astype(np.float32)  
output = session.run(None, {input_name:x})  
print(output)
```

Run:

```
python Python\onnx_inference.py
```

Final Verification

You should verify:

-  ONNX installed
-  ONNX Runtime installed
-  ONNX Simplifier installed
-  Netron available (web or local)
-  First ONNX model exported
-  ONNX model validated
-  ONNX Runtime inference executed
-  ONNX graph visualized

11 – Installing and Configuring STM32CubeIDE 1.19.0

Step 1

Open a web browser.

Visit the STMicroelectronics website www.st.com

Search for:

STM32CubeIDE

or directly navigate to the STM32CubeIDE download page.

Step 2

Download

STM32CubeIDE **1.19.0**

Select the installer for Windows (64-bit).

Step 3

Log in using the ST account created earlier

Verification

Participants should have downloaded an installer similar to:

`en.stm32cubeide-win64-1.19.0.exe`

Step 4

Run the installer.

Follow the installation wizard.

Recommended options:

- Accept the license agreement.
- Install all default components.
- Keep the default installation directory.
- Install device drivers if prompted.

Verification

After installation:

A desktop shortcut and/or Start Menu entry for **STM32CubeIDE** should be available.

Step 5 – First Launch

Launch STM32CubeIDE.

The first launch may take a minute while the IDE initializes.

Workspace Selection

Participants will be prompted to choose a workspace.

Recommend creating:

D:\STM32_Workspace

or

C:\Users\<username>\STM32_Workspace

Enable:

Use this as the default and do not ask again

Click:

Launch

Step 6 – Create the First STM32 Project

Select:

File → New → STM32 Project

The **Target Selection** window appears.

Select Board

Choose:

Board Selector

Search:

B-L4S5I-IOT01A

Click:

Next

Project Settings

Project Name:

First_STM32_Project

Project Location:

Leave the default workspace location.

Click:

Finish

Verification

The project appears in the Project Explorer.

Step 7 – Explore the Device Configuration Tool

When the project opens, the .ioc file is displayed automatically.

Explore tabs:

- Pinout & Configuration
- Clock Configuration
- Project Manager
- Software Packs

Step 8 – Generate Initialization Code

Click:

Project → Generate Code

STM32CubeIDE generates:

- Startup files
- HAL drivers
- Peripheral initialization
- Linker scripts
- Project structure

Verification

The Project Explorer should now contain:

```
Core
Drivers
STM32CubeIDE
Startup
Middlewares (if enabled)
First_STM32_Project.ioc
```

Step 9 – Build the Project

Select:

Project → Build Project

or click the hammer icon.

Verification

The Build Console should report something similar to:

Build Finished

0 Errors,

0 Warnings

Final Verification Checklist

Each participant should confirm:

- ☒ STM32CubeIDE 1.19.0 installed.
- ☒ Workspace created.
- ☒ IDE launches successfully.
- ☒ B-L4S5I-IOT01A board selected.
- ☒ First project created.
- ☒ Device Configuration Tool opens.
- ☒ Code generated successfully.
- ☒ Project structure created.
- ☒ Project builds successfully.

Step 10 – Installing and Configuring X-CUBE-AI

Launch STM32CubeIDE.

Go to:

Help



Manage Embedded Software Packages

Step 11 – Install X-CUBE-AI

In the package manager:

Search for:

X-CUBE-AI

Select the latest stable version compatible with STM32CubeIDE 1.19.0.

Click:

Install

Accept the license agreement.

Wait for the installation to complete.

Verification

Participants should observe:

“Installed” next to X-CUBE-AI.

Step 12 – Verify Installation

Close the package manager.

Create a new STM32 project (or use the one created earlier).

Open the .ioc file.

Verify that the **Software Packs** tab includes:

X-CUBE-AI

This confirms the package is available.

Step 13 – Enable X-CUBE-AI

Inside the .ioc file:

Go to:

Software Packs

Expand:

X-CUBE-AI

Enable:

Artificial Intelligence

Save the project.

STM32CubeIDE regenerates the project.

Verification

Observe new project folders such as:

```
X-CUBE-AI/  
Network/
```

(depending on the version).

Step 14 – Import an ONNX Model

Use the ONNX model created earlier:

```
simple_model.onnx
```

Browse and select the model.

Click:

Analyze

Verification

You should observe information similar to:

- Number of layers
- Number of parameters
- MACC (Multiply–Accumulate Operations)
- Flash memory requirement
- RAM requirement

Step 15 – Understanding the Analyzer

Discuss the generated statistics:

Metric	Meaning
Parameters	Number of trainable weights
MACC	Computational complexity
Flash	Memory required for weights and code
RAM	Runtime memory usage
Activation Memory	Intermediate tensors
Runtime Memory	Total memory required during inference

Step 16 – Generate AI Code

Click:

Generate Code

STM32CubeIDE creates optimized C source files.

Verification

Participants should observe generated files such as:

```
app_x-cube-ai.c  
app_x-cube-ai.h  
network.c
```

network.h

(File names may vary slightly depending on the X-CUBE-AI version.)

These files implement the neural network in C.

Step 17 – Build the Project

Build the project.

Project



Build Project

Verification

You should observe:

Build Finished

0 Errors,

0 Warnings

Final Verification Checklist

Each participant should verify:

- ☒ X-CUBE-AI installed.
- ☒ X-CUBE-AI visible in Software Packs.
- ☒ AI middleware enabled.
- ☒ ONNX model imported.
- ☒ Model analyzed successfully.
- ☒ Flash and RAM usage displayed.
- ☒ AI source code generated.
- ☒ Project builds successfully.

12 – Development Environment Verification

Step 1 - Verify Python Environment

python --version

pip --version

python Python\scientific_stack_test.py

python Python\deep_learning_stack_test.py

Expected:

Scientific Python Environment Ready!

Deep Learning Environment Ready!

Step 2 – Verify ONNX Environment

Run

python Python\verify_onnx.py

python Python\verify_onnxruntime.py

python Python\export_onnx.py

Verify:

simple_model.onnx
exists.

Step 3 – Verify STM32CubeIDE

Open

First_STM32_Project

Build.

Expected:

Build Finished

0 Errors

Step 4 – Verify X-CUBE-AI

Open

simple_model.onnx

Analyze.

Generate.

Build.

Step 5 – Verify Discovery Kit

Connect the board.

Click

Run

or

Debug

Verify

- Download successful.
- Debug session starts.
- Stop debugging.

13 – Environment Health Checklist

Component	Status
VS Code	☐
Git	☐
GitHub	☐
Python	☐
Virtual Environment	☐
NumPy	☐
Pandas	☐
Matplotlib	☐
SciPy	☐
Jupyter	☐
PyTorch	☐
TorchVision	☐
TorchAudio	☐
ONNX	☐
ONNX Runtime	☐
ONNX Simplifier	☐
ONNX Script	☐
Netron	☐
STM32CubeIDE	☐
X-CUBE-AI	☐
B-L4S5I-IOT01A Board	☐
Project Build	☐
Debug Session	☐